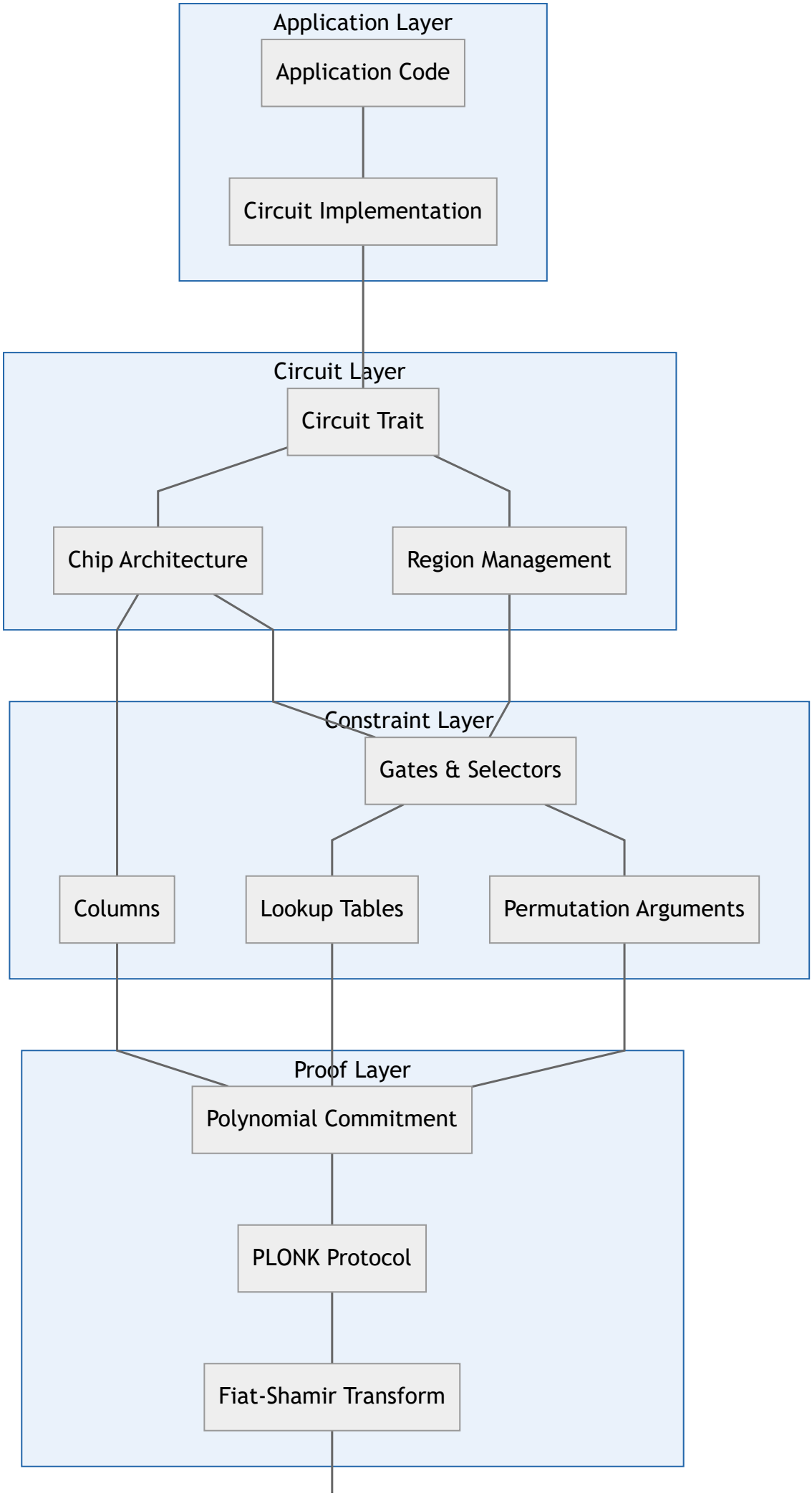
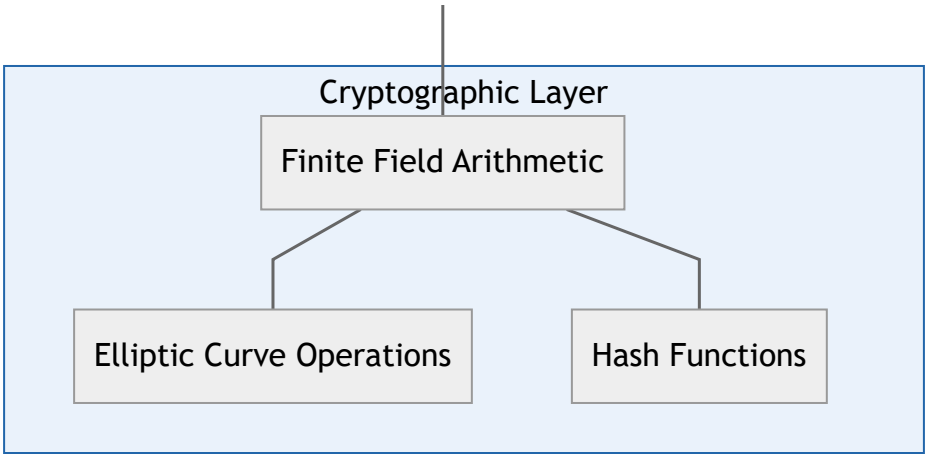
halo2 아키텍처

전체 아키텍처 개요

halo2는 영지식 증명 시스템을 구현하기 위한 계층화된 아키텍처를 가지고 있습니다. 각 계층은 명확한 역할과 책임을 가지며, 상위 계층은 하위 계층의 기능을 활용하여 더 복잡한 기능을 제공합니다.


Application Layer
Application Code
Circuit Implementation
Circuit Layer
Circuit Trait
Chip Architecture
Region Management
Constraint Layer
Gates & Selectors
Columns
Lookup Tables
Permutation Arguments
Proof Layer
Polynomial Commitment
PLONK Protocol
Fiat-Shamir Transform



주요 컴포넌트

1. Circuit Layer (회로 계층)

Circuit Trait

- **역할**: 증명하고자 하는 계산을 회로로 정의하는 인터페이스
- **주요 메서드**:
- `configure()` : 회로의 제약조건과 구조 정의
- `synthesize()` : 실제 값들을 회로에 할당
- `without_witnesses()` : 증명 키 생성을 위한 회로 구조만 제공

```
trait Circuit<F: Field> {
type Config: Clone;
type FloorPlanner: FloorPlanner;
fn  configure(meta: &mut ConstraintSystem<F>) -> Self::Config;
fn  synthesize(&self, config: Self::Config, layouter: impl Layouter<F>) -> Resul
fn  without_witnesses(&self) -> Self;
}
```

Chip Architecture

- **역할**: 재사용 가능한 회로 컴포넌트 제공
- **특징**:
- 모듈화된 설계로 코드 재사용성 향상
- 복잡한 회로를 작은 단위로 분해
- 각 Chip은 특정 기능에 특화

Region Management

- **역할**: 회로 내에서 메모리 영역 관리
- **기능**:
- 효율적인 메모리 사용
- 제약조건 적용 범위 제어
- 병렬 처리 최적화

2. Constraint Layer (제약조건 계층)

Gates & Selectors

- **Gate**: 특정 제약조건을 나타내는 다항식
- **Selector**: Gate의 활성화/비활성화를 제어
- **관계**: Selector가 활성화된 행에서만 Gate가 적용됨

```
// 예시: 덧셈 Gate
// a + b = c 제약조건
let  gate = meta.create_gate("addition", |meta| {
let  a = meta.query_advice(advice[0], Rotation::cur());
```

```
let  b = meta.query_advice(advice[1], Rotation::cur());

let  c = meta.query_advice(advice[2], Rotation::cur());

let  s = meta.query_selector(selector);

vec![s * (a + b - c)]

});
```

Column Types

1. **Advice Columns**: 증명자가 제공하는 비공개 값들
2. **Instance Columns**: 공개적으로 알려진 값들
3. **Fixed Columns**: 회로 설계 시 고정된 상수들

Lookup Tables

- **역할**: 복잡한 연산을 미리 계산된 테이블로 대체
- **장점**: 복잡한 제약조건을 효율적으로 처리
- **종류**:
- Static Lookup: 컴파일 시 고정된 테이블
- Dynamic Lookup: 실행 시 생성되는 테이블

3. Proof Layer (증명 계층)

Polynomial Commitment

- **역할**: 다항식에 대한 암호학적 약속 생성
- **기능**:
- 다항식을 공개하지 않고 약속 생성
- 특정 점에서의 값 증명

- 다항식 연산 결과 검증

PLONK Protocol

- **역할**: 실제 영지식 증명 생성과 검증
- **단계**:

1. **Setup**: 공통 참조 문자열 생성
2. **Prove**: 증명 생성
3. **Verify**: 증명 검증

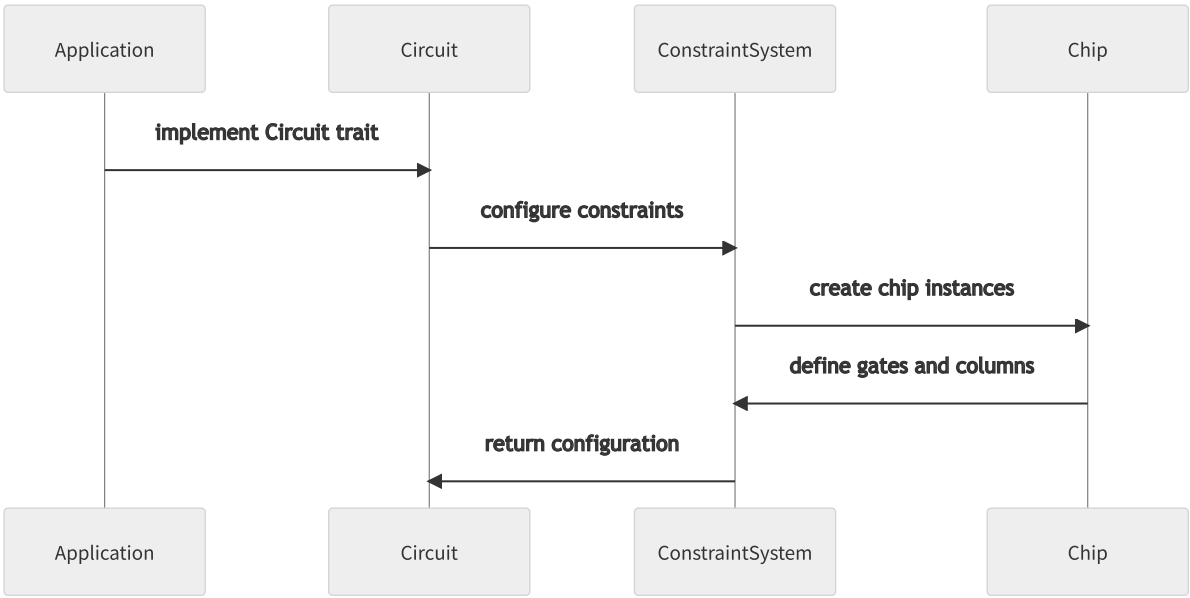
4. Cryptographic Layer (암호학 계층)

Finite Field Arithmetic

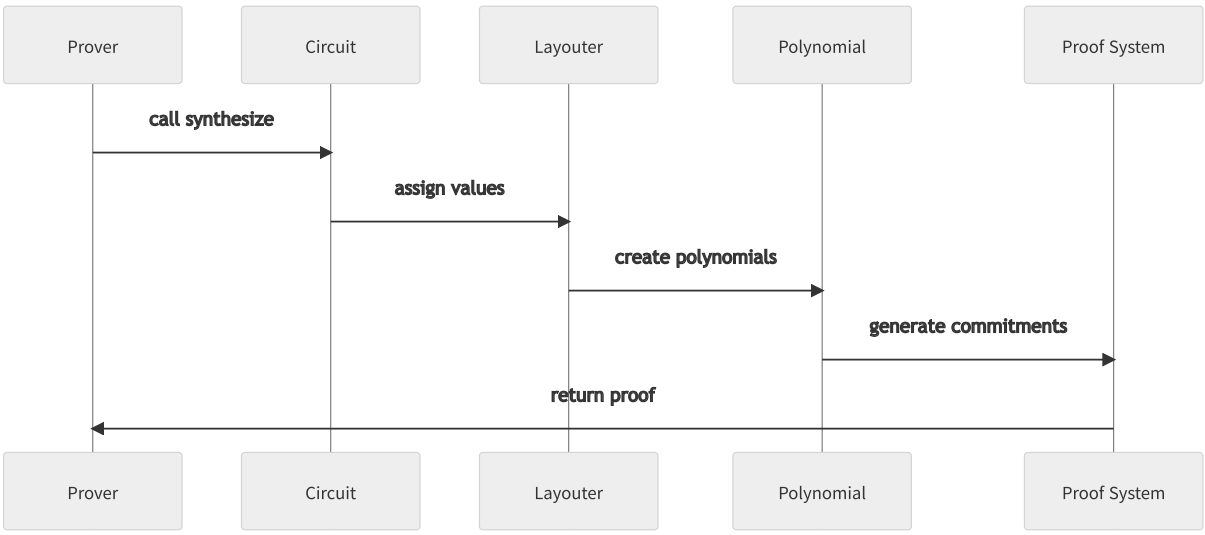
- **역할**: 모든 연산의 기반이 되는 유한체 연산
- **특징**:
- 고정된 크기의 정수 연산
- 모듈러 산술 연산
- 효율적인 구현을 위한 최적화

컴포넌트 간 상호작용

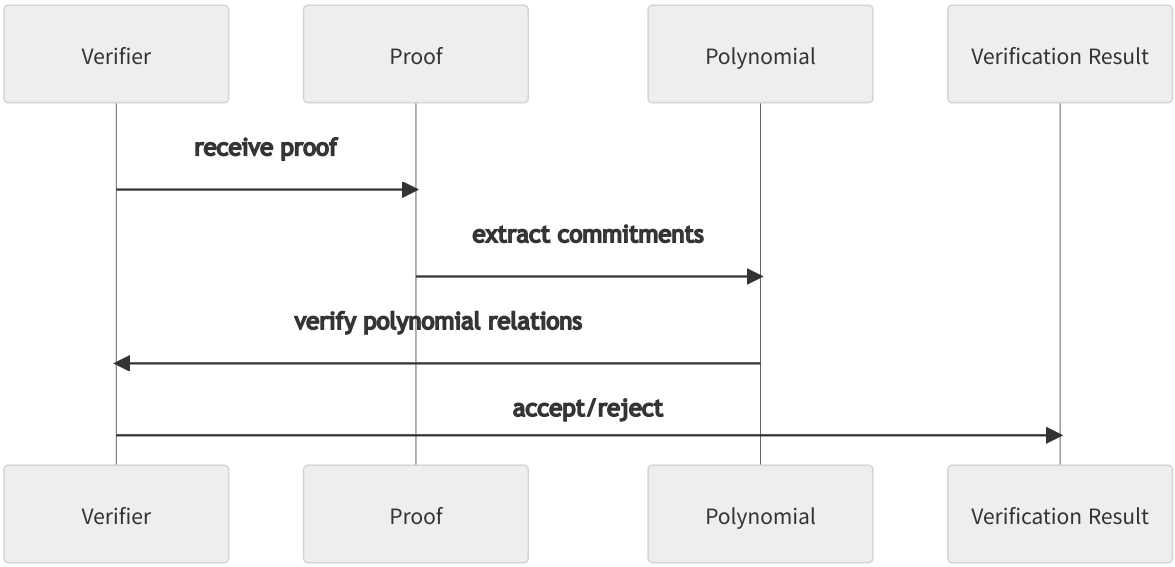
1. Circuit 정의 과정


Application
Circuit
ConstraintSystem
Chip
implement Circuit trait
configure constraints
create chip instances
define gates and columns
return configuration
Application
Circuit
ConstraintSystem
Chip

2. Proof 생성 과정


Prover
Circuit
Layouter
Polynomial
Proof System
call synthesize
assign values
create polynomials
generate commitments
return proof
Prover
Circuit
Layouter
Polynomial
Proof System

3. Proof 검증 과정



메모리 구조

Column Layout

halo2는 테이블 형태의 메모리 구조를 사용합니다:

```
Row | Advice | Instance | Fixed | Selector
----|----------|------------|----------|----------
0   |   a₀, b₀  |     c₀     |    k₀    |    s₀
1   |   a₁ , b₁ |     c₁     |    k₁    |    s₁
2   |   a₂ , b  |     c₂     |    k₂    |    s₂
... | ...       | ...        | ...      | ...
```

Region 할당

- **Sequential Region**: 순차적으로 행을 할당
- **Parallel Region**: 병렬로 여러 영역 동시 사용
- **Nested Region**: 중첩된 영역 구조

최적화 전략

1. Circuit 최적화

- **Gate 최적화**: 불필요한 제약조건 제거
- **Column 재사용**: 메모리 사용량 최소화
- **Lookup 활용**: 복잡한 연산을 테이블로 대체

2. Proof 최적화

- **Batch Verification**: 여러 증명을 한 번에 검증
- **Preprocessing**: 반복되는 계산 사전 처리
- **Parallel Processing**: 병렬 처리를 통한 성능 향상

3. Memory 최적화

- **Streaming**: 대용량 데이터의 스트리밍 처리
- **Caching**: 자주 사용되는 값들의 캐싱
- **Garbage Collection**: 불필요한 메모리 해제

확장성 고려사항

1. 수평적 확장

- **Sharding**: 회로를 여러 부분으로 분할

- **Aggregation**: 여러 증명을 하나로 집계
- **Recursive Proof**: 증명의 증명을 통한 확장

2. 수직적 확장

- **Hardware Acceleration**: GPU/FPGA 활용
- **Algorithm Optimization**: 더 효율적인 알고리즘 적용
- **Memory Hierarchy**: 다층 메모리 구조 활용

보안 고려사항

1. 암호학적 보안

- **Discrete Log Problem**: 타원곡선 이산로그 문제의 어려움에 의존
- **Knowledge Soundness**: 증명자가 실제로 witness를 알고 있음을 보장
- **Zero-Knowledge**: 증명 과정에서 정보 누출 방지

2. 구현 보안

- **Side-Channel Attack**: 부채널 공격 방지
- **Constant-Time Operation**: 타이밍 공격 방지
- **Memory Safety**: Rust의 메모리 안전성 활용

다음 단계

아키텍처를 이해했다면 다음 문서들을 학습하세요:

- PLONK와의 관계: 이론적 배경 이해
- 다른 ZK 시스템과의 비교: halo2의 특징 파악

- 기본 개념: 구체적인 구현 개념 학습

> **참고**: 이 아키텍처는 halo2의 전체적인 구조를 보여줍니다. 각 컴포넌트의 구체적인 사용법은 해당 섹션에서 자세히 다룹니다.